ARGUS TECHNICAL REPORT 0.3 · JULY 2026

Argus: Long-Horizon Autonomous Research through Persistent, Evidence-Gated Runtime Evolution

Shanghai Jiao Tong University **Microsoft**

Shanghai Jiao Tong University

Microsoft

Abstract. Long-horizon autonomous research requires more than extending a single model session. The system must preserve objectives across restarts, turn decisions into real experiments, separate execution from acceptance, and retain verified lessons for later work. We present ARGUS, a persistent runtime for autonomous research organized around four model-driven roles—Manager, Planner, Engineer, and Reviewer—operating over bounded missions and durable project state. The runtime records an append-only evidence trace, maintains curated cross-session memory, and updates reusable state such as skills, procedures, routing, and evaluations while holding the underlying model parameters fixed. We describe this mechanism as evidence-gated runtime evolution: a mission changes the operating state only through an attributable update grounded in artifacts and review. We assess the system using two public evidence sources: results from six heterogeneous benchmark arenas and a de-duplicated portfolio of 41 research artifacts across six programs. Two benchmark results include committed artifact-digest metadata; four are currently supported by public website snapshots. The evidence demonstrates that the runtime can sustain productive work across distinct research settings, but it does not isolate the causal effect of individual architectural components or establish universal state-of-the-art performance. We therefore report protocols, provenance tiers, and limitations alongside every aggregate claim.

Keywords: autonomous research agents; long-horizon agents; persistent state; evidence-grounded evaluation; agent reliability.

Code: github.com/lbx154/argus-skill

This is a living technical report. Reported results and evidence tiers correspond to the public snapshot retrieved on July 14, 2026.

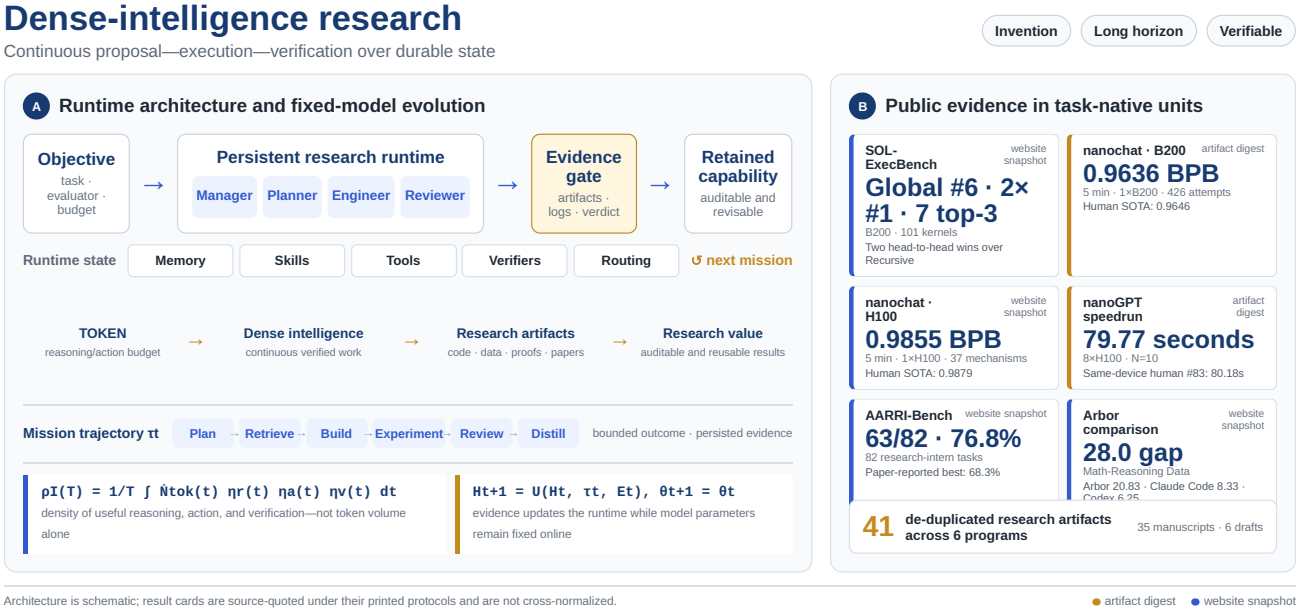

Figure 1. ARGUS at a glance. (A) A dense-intelligence objective enters a role-separated runtime; artifacts and review evidence pass through an evidence gate before updating reusable state around a fixed model. (B) The current public record spans six heterogeneous arenas and 41 research artifacts. Result cards preserve task-native units and evidence tiers; they are not cross-normalized.

1 Introduction

Scientific research is a long-horizon activity. A useful result usually emerges from a sequence of dependent decisions: selecting a direction, modifying an artifact, running an experiment, interpreting the evidence, and deciding what to try next. The difficulty is therefore not only whether a model can solve an isolated reasoning problem, but whether a system can preserve the state of inquiry across many model calls, tool invocations, failures, restarts, and delayed measurements.

Large language model agents have made substantial progress on tool use and software tasks. Toolformer learns when to invoke external tools (Schick et al., 2023), ReAct couples reasoning with actions (Yao et al., 2023), and systems such as SWE-agent and OpenHands expose repository editing through agent-computer interfaces (Yang et al., 2024; Wang et al., 2024a). Automated research systems extend this idea to experiment generation and scientific writing (Lu et al., 2024; Jiang et al., 2025; Yamada et al., 2025), while Arbor represents research as a persistent hypothesis tree refined by a coordinator and isolated executors (Jin et al., 2026). These systems show that models can carry out increasingly complex research actions. They also make the remaining systems problem more visible: longer execution does not by itself make the work cumulative, auditable, or reliable.

In practice, long-running agents fail in three recurrent ways, consistent with evidence that agent success declines as software-task horizons grow (Kwa et al., 2025). First, *continuity* is fragile: a growing transcript is eventually compacted or dropped, and later calls spend time reconstructing decisions that were already made. Second, *acceptance* is ambiguous when the component that performs the work also declares it complete. Third, *experience* is lost when only the final artifact survives; failed branches, contaminated measurements, and useful procedures disappear with the session that produced them. Hierarchical and workflow-oriented memory systems address parts of this problem (Packer et al., 2023; Wang et al., 2024b; Xu et al., 2025). A larger context window can postpone the first failure, but it does not resolve the other two and does not turn a transcript into durable institutional memory.

We present ARGUS (Autonomous Research Generation and Understanding System), a persistent runtime for long-horizon autonomous research. Its central abstraction is a sequence of *bounded missions* executed against a durable project state. Four model-driven roles divide authority: a Manager anchors the objective and campaign state, a Planner selects the next units of work, an Engineer implements and evaluates them, and a Reviewer inspects artifacts and evidence before issuing the completion verdict. These roles operate across three planes—control, execution, and evidence—so that scheduling, work, and record keeping remain separable even though they participate in one continuous loop.

The runtime is designed to accumulate useful experience without updating the base model. Each mission emits an attributable trajectory and a set of accepted evidence. The system uses these records to update persistent memory, reusable skills, procedures, verifiers, routing, and evaluations while holding the model parameters fixed. We call this mechanism *evidence-gated runtime evolution*. The term *dense intelligence* describes the corresponding design goal: keep decision, execution, and verification connected over the research horizon rather than concentrating useful reasoning into isolated bursts. It is a design principle, not a reported scalar metric.

We study the resulting system through two public evidence sources. The first contains six heterogeneous benchmark arenas spanning GPU kernels, language-model training, research-assistant tasks, and data synthesis. The second is a de-duplicated inventory of 41 research artifacts across six programs. The evidence is intentionally tiered. Two benchmark results include committed artifact-digest metadata, whereas four currently remain public website snapshots. The results show that the runtime can support productive work across distinct settings, but the current evaluation is observational: it does not isolate the effect of each architectural component under a common controlled baseline.

This report makes three contributions:

C1. We formulate long-horizon autonomous research as a sequence of bounded missions over persistent state, separating artifact improvement from the retention and verification mechanisms that make progress cumulative.

C2. We introduce a role-separated runtime that combines durable objectives, bounded context, independent review, append-only evidence, and fixed-model runtime evolution in one operational system.

C3. We provide a protocol-explicit empirical record across six benchmark arenas and six research programs, preserving evidence tiers and claim boundaries instead of collapsing heterogeneous results into a single score.

The remainder of the report follows a conventional research-paper organization. Section 2 positions the system against prior agent and autonomous research work. Section 3 formalizes the operating problem; Section 4 presents the runtime; Sections 5 and 6 describe the evidence protocol and results; and Sections 7–9 discuss interpretation, limitations, and conclusions. Implementation contracts and operational defaults are moved to the appendix.

2 Related Work

2.1 Tool-Using and Software-Engineering Agents

Language-agent research established the core loop of selecting actions, invoking external tools, and conditioning later decisions on observations. Toolformer learns when to call tools from self-supervised data (Schick et al., 2023), while ReAct interleaves textual reasoning with environment actions (Yao et al., 2023). For repository-scale work, SWE-agent emphasizes the importance of an agent-computer interface (Yang et al., 2024), and OpenHands provides an open platform for generalist software-development

agents (Wang et al., 2024a).

Evaluation has moved correspondingly from isolated code generation toward interactive tasks. SWE-bench draws problems from real GitHub issues (Jimenez et al., 2024); AgentBench evaluates agents across heterogeneous interactive environments (Liu et al., 2023); and AgentBoard adds fine-grained progress analysis rather than reporting only final success (Ma et al., 2024). Time-horizon studies further show that performance on software tasks degrades as task duration increases (Kwa et al., 2025). These results motivate treating long-horizon capability as a property of the complete runtime, not only the model that generates the next action.

2.2 Persistent Memory and Experience Reuse

Several lines of work externalize experience beyond a single context window. Generative Agents combine stored observations, reflection, and retrieval for multi-day simulated behavior (Park et al., 2023). Reflexion retains verbal feedback across attempts (Shinn et al., 2023), and Voyager accumulates an executable skill library during open-ended interaction (Wang et al., 2023). MemGPT frames context management as a hierarchy of memory tiers (Packer et al., 2023). More recent systems induce reusable task workflows (Wang et al., 2024b) or organize memories through agentic linking and retrieval (Xu et al., 2025).

ARGUS shares the premise that useful experience should survive a model call, but couples retention to explicit authority and evidence. The Engineer may propose a checkpoint or procedure; the Reviewer commits the canonical retained form after inspecting artifacts and execution evidence. This makes memory not only retrievable but also attributable, versioned, and reversible.

2.3 Autonomous Research and Program Search

Automated research systems increasingly connect idea generation, implementation, experimentation, and scientific communication. The AI Scientist presents an end-to-end scientific-discovery pipeline (Lu et al., 2024); Agent Laboratory organizes specialized agents as research assistants (Schmidgall et al., 2025); and AIDE explores machine-learning improvements through iterative code search (Jiang et al., 2025). The AI Scientist-v2 introduces agentic tree search over research plans and experiments (Yamada et al., 2025). Arbor similarly uses a persistent hypothesis tree, a long-lived coordinator, and isolated executors to make research cumulative (Jin et al., 2026).

Executable evaluation can also drive discovery without reproducing a conventional scientific workflow. FunSearch evolves programs against objective functions and has produced new mathematical constructions (Romera-Paredes et al., 2024); AlphaEvolve extends evolutionary code improvement to scientific and systems problems (Novikov et al., 2025). These systems demonstrate the value of a hard evaluator and repeated proposal–execution–selection cycles. ARGUS focuses on the more general runtime substrate around such cycles: persistent intent, bounded missions, evidence admission, restartability, and reusable state across heterogeneous research programs.

2.4 Benchmarks for Research Agents

Research-agent evaluation is beginning to cover realistic machine-learning and scientific work. MLE-bench packages Kaggle competitions as machine-learning engineering tasks (Chan et al., 2024); RE-Bench compares agents and human experts in open-ended research-engineering environments (Wijk et al., 2024); AARRI-Bench evaluates research-lifecycle tasks for models and harnesses (Wang et al., 2026); and AIRS-Bench targets frontier research-science agents across a suite of scientific tasks (Lupidi et al., 2026). Together these benchmarks emphasize held-out evaluation, task-native metrics, trajectories, and resource controls.

The present report is complementary but weaker as a causal evaluation. It aggregates public results produced under different protocols and clearly distinguishes artifact-linked evidence from website snapshots. This supports claims about operational breadth and individual task outcomes, while leaving common-budget comparisons and component ablations to future work.

3 Problem Formulation

3.1 Dense-Intelligence Tasks

The public Argus formulation defines a *dense-intelligence task* as one that sustains high-frequency reasoning, tool use, verification, and iteration across a continuous time window until it produces an auditable result (Argus Team, 2026d). Three conditions determine whether a task has this shape:

T1. Invention. The answer is not directly retrievable from a manual or database; it must be discovered through search and feedback.

T2. Long horizon. One pass is insufficient. Progress requires many dependent iterations over hours or days.

T3. Verifiability. A task-native evaluator can distinguish genuine improvement from a persuasive but incorrect claim.

The defining loop is therefore proposal → execution → measurement → revised proposal. Performance engineering, security research, scientific search, and quantitative research differ in domain but share this operational structure. The task definition excludes both open-ended ideation without an evaluator and fixed workflows that require time but no new decisions.

We model a campaign by an objective o, an initial artifact y_0, an evaluation procedure f, and a resource budget B. The artifact

may be code, a model, a dataset, an experimental harness, a proof, or a manuscript. The campaign is divided into bounded missions indexed by t, each producing

$$\tau_t = (s_t, a_t, y_t, e_t, r_t), \tag{1}$$

where s_t is the mission state, a_t the actions taken, y_t the resulting artifact state, e_t the observed evidence, and r_t the review outcome. A mission must terminate, pause, or transfer control explicitly; continuity belongs to the persistent campaign state rather than an unbounded model session.

3.2 Dense-Intelligence Density

The website expresses useful intelligence per unit wall-clock time as

$$\rho_I(T) = \frac{1}{T}\int_0^T \dot{N}_{\text{tok}}(t)\,\eta_r(t)\,\eta_a(t)\,\eta_v(t)\,dt. \tag{2}$$

$\dot{N}_{\text{tok}}(t)$ is the instantaneous rate of token-mediated reasoning and action. The factors $\eta_r(t)$, $\eta_a(t)$, and $\eta_v(t)$ represent the fractions converted into relevant reasoning, effective action, and valid verification. The product is intentionally stricter than token throughput: a run that repeatedly reads the same state, emits activity without changing an artifact, or accepts unverified claims can consume many tokens while contributing little to ρ_I.

The public page summarizes the intended continuous-utilization advantage over a fixed 48-hour window as

$$\rho_I^{\text{Argus}}(48\text{h}) \gg \rho_I^{\text{human}}(48\text{h}). \tag{3}$$

Equation 3 is a public design hypothesis, not a measured result in this report. It refers to the density of sustained work over continuous time, not to unconditional superiority of any individual reasoning step. Neither ρ_I nor its conversion factors are instrumented as a benchmark score in the current evaluation.

3.3 Online Life and Runtime Evolution

The second website object separates model parameters from the operating state around the model:

$$\begin{aligned} H_{t+1} &= U(H_t, \tau_t, E_t), \\ \theta_{t+1} &= \theta_t. \end{aligned} \tag{4}$$

with the public state definition

$$H_t = \{\text{Memory}, \text{Skills}, \text{Tools}, \text{Verifiers}, \text{Routing}\}. \tag{5}$$

E_t is the subset of evidence admitted from mission t. The equality for θ is a scope condition: online evolution does not require a gradient update to the underlying model. Section 4 refines H_t into named ownership surfaces and tracks task/evaluation definitions as an additional component Q_t. The update U is partial; many missions change only memory, and some change no reusable component.

3.4 Evidence-Gated OOD Expansion

Let C_t denote the effective capability set available at time t. The website writes evidence-gated admission and the desired breadth/depth behavior as

$$\begin{aligned} &C_{t+1} = C_t \cup \{c : \text{Verify}(c, E_t) \geq \varepsilon\}, \\ &W_{t+1} \geq W_t, \qquad\qquad D_{t+1}(d) \geq D_t(d). \end{aligned} \tag{6}$$

where W_t is frontier width across domains and $D_t(d)$ is depth within domain d. The inequalities are retention desiderata over verified capability, not an empirical guarantee that every run improves the system. Skills can become stale, verifiers can be revised, and performance can regress on harder instances. A negative result can still add value by excluding a failed branch even when the effective capability set does not grow.

3.5 Research Value

Dense activity matters only when it produces new, valid, reusable information. The website therefore defines the accumulated research value over horizon T as

$$V_R(T) = \int_0^T \rho_I(t)\,\Delta I(t)\,p_{\text{valid}}(t)\,\eta_{\text{reuse}}(t)\,dt, \tag{7}$$

where $\Delta I(t)$ is information gain, $p_{\text{valid}}(t)$ is the probability that the claimed increment survives verification, and $\eta_{\text{reuse}}(t)$ is the fraction that remains useful for later work. Equation 7 explains why token volume alone is not the product: research value

collapses if the work is repetitive, invalid, or non-reusable. It is a conceptual objective, not a calibrated monetary or benchmark metric in this report.

3.6 The Information Advantage of Process Data

Finally, the public formalization distinguishes the accepted artifact from the trajectory that produced it:

$$D_{\text{process}} = \{(s_k, a_k, e_k, r_k, \Delta H_k)\}_{k=1}^{N} \supsetneq D_{\text{final}} = \{y^\star\}. \tag{8}$$

The process record contains states, actions, evidence, review feedback, and runtime-state changes, including failed branches omitted from $y^\star$. This additional information can reduce repeated search, support audit, and provide grounded material for later skill construction or evaluation. ARGUS retains typed, credential-redacted observations and verdicts, not private chain-of-thought transcripts.

4 Argus Runtime

4.1 System Overview

ARGUS is an open-source persistent research runtime (Argus Team, 2026a) built around four model-driven roles and three system planes. The *control plane* anchors the campaign and schedules work; the *execution plane* performs one bounded mission against real tools and artifacts; and the *evidence plane* records what happened without deciding whether the work is scientifically complete. The distinction is operational: control owns scheduling, execution owns work and mission review, and evidence owns the immutable record but no completion decision.

The four roles divide research authority as summarized in Table 1. The Manager spans the campaign rather than one execution round. The Planner authors future work, the Engineer performs it, and the Reviewer is the sole mission-level completion authority. The interfaces are structured objects rather than untyped prose, which makes ownership and recovery behavior explicit.

Role	Primary responsibility	Authoritative output
Manager	Commit the objective, lifetime, route, and campaign stage	Campaign division and stage transition
Planner	Decompose the current research state into bounded tasks and dependencies	Task specifications, plan status, and stage checklist updates
Engineer	Modify artifacts, invoke tools, and run experiments for one round	Execution result, artifacts, measurements, and a proposed handoff
Reviewer	Inspect artifacts and execution evidence; decide whether work is complete	`done`, `continue`, or `blocked`, plus grounded next action and retained-state edits

Table 1. Role separation in ARGUS. The role that performs the work does not own the completion verdict.

Algorithm 1: Evidence-gated mission loop

Input: objective o, runtime state H_t, backlog B, budget b
$c \leftarrow \text{ManagerCommit}(o)$; persist campaign identity
while b remains and c is not complete **do**
  $q \leftarrow \text{Planner}(B, H_t, c)$; claim one bounded task
  $\tau_t \leftarrow [\,]$
  repeat
    $(y, e) \leftarrow \text{Engineer}(q, H_t)$
    $r \leftarrow \text{Reviewer}(q, y, e)$; append (q, y, e, r) to τ_t
    if $r =$ `continue` **then** $q \leftarrow r$.`next_action`
  until $r \in \{$`done`, `blocked`, `paused`$\}$
  $E_t \leftarrow \text{AdmitEvidence}(e, r)$
  $H_{t+1} \leftarrow U(H_t, \tau_t, E_t)$; persist outcome, evidence, and state
  $t \leftarrow t + 1$; refill B when required
end while
Output: accepted artifacts, auditable trajectory, and updated runtime state

4.2 Bounded Missions over Durable State

The long-lived object in ARGUS is the campaign, not a provider transcript. A campaign has a persisted identity and objective; its work is divided into missions that have explicit outcomes. The outer supervisor claims one backlog item at a time, executes it,

records the result, and advances only at a clean mission boundary. Atomic claims prevent two workers from taking the same item, while identity-checked resume prevents a restarted process from attaching to the wrong campaign.

Model sessions are treated as bounded caches. Engineer and Reviewer sessions can be resumed briefly to preserve useful local context, but they are rolled at explicit limits. Cross-session continuity comes from a curated checkpoint containing the goal, verified progress, failed approaches, the current blocker, and the next step. This follows the broader move from monolithic context toward managed memory tiers and reusable workflows (Packer et al., 2023; Wang et al., 2024b; Xu et al., 2025). The Engineer proposes the handoff; the Reviewer validates and writes the canonical checkpoint. Hard size limits keep this object focused on current state, while the full history remains in artifacts and the event record. Default roll thresholds and the complete state layout are reported in Appendix B.

This design makes restart and upgrade behavior part of the method. A daemon drains at a mission boundary, transfers campaign identity to a replacement process, and resumes only after checking that identity. A crash may require replay or re-claiming a mission, but it does not require reconstructing the campaign from a model transcript.

4.3 Evidence-Gated Completion

Every provider call passes through one application-level gateway that assigns a call identifier, records timing and usage presence, and binds the call to the mission event stream. The Engineer and Reviewer share the mission worktree, so the Reviewer can inspect the actual artifacts and execution log rather than relying on the Engineer's summary. A typed, append-only event tape is the canonical timeline; mutable status files and cockpit views are projections over that record.

The Reviewer is the sole source of a mission completion verdict. The infrastructure does not infer completion from filenames, token volume, or keywords, and it does not silently rewrite the Reviewer's decision. A successful verdict may still be wrong because the Reviewer is a model, but the authority is unambiguous and the evidence used by the decision is auditable. This is a traceability guarantee, not a guarantee of scientific correctness.

Evidence is sanitized before persistence. A task-agnostic secret guard redacts credential patterns from event and artifact copies supplied to later roles. If an artifact is rewritten by redaction, the event stream records the intervention so affected provenance can be rebuilt. Public result evidence is separately labeled as a website snapshot or artifact-digest corroboration; Section 5 defines those tiers.

4.4 Runtime-State Updates

Equation 4 separates the model from the state around it. Table 2 identifies how each component can change and who owns the committed update. The ownership model is intentionally not uniform. Memory and skills use a work-versus-certification split: the Engineer or Scientist produces a candidate, and the Reviewer commits the retained form. Tools remain operator-owned. Stage checklists are Planner-owned, with Reviewer feedback rather than a second commit gate. Routing is operator-sourced and Manager-committed, while task definitions are Planner-authored and scheduler-committed.

State	Change source	Commit owner	Persistent form
M: memory	Engineer trajectory	Reviewer	Curated checkpoint and event-derived journal
S: skills	Engineer / Scientist	Reviewer	Versioned skill library
A: tools and procedures	Operator / vertical implementation	Operator	Installed tools and stage procedures
V: verification	Planner	Planner	Stage checklist; Reviewer supplies feedback
R: routing and roles	Operator	Manager	Campaign configuration and knob store
Q: tasks and evaluations	Planner	Scheduler	Backlog task specifications and scopes

Table 2. Attributable ownership of fixed-model runtime evolution. A mission usually updates only a subset of these components.

Two knowledge surfaces carry reusable experience. Versioned skills store procedures that can be matched to later tasks. A project wiki stores immutable source cards derived from real Reviewer verdicts and may later synthesize evidence-cited pages. Neither surface is treated as automatically correct: entries can be revised, archived, or retired when later evidence contradicts them.

4.5 Reliability and Resource Governance

Long-running systems fail through operational drift even when individual model calls are strong. ARGUS therefore wraps the mission loop with task-agnostic guards. The Reviewer labels whether a round produced a decision, evidence, setup only, artifact synchronization only, or no progress. Round-boundary guards stop or re-plan a mission after sustained non-decision behavior; in-round watchdogs are reserved for subprocesses that are idle or emitting activity without effective progress. These mechanisms bound execution but never decide whether a scientific claim is correct.

The same gateway that records calls also enforces reservation-backed budgets. Spend is reserved before a call, reconciled

afterward, and released on abort. Provider-side spend fences are derived from the reservation rather than role configuration, so a role cannot raise its own limit. Long-running external jobs are registered and monitored separately; the Engineer can explicitly wait for a healthy supervised job instead of repeatedly polling it. GPU leasing, sandboxing, daemon handoff, cockpit authentication, and the numerical defaults for all guards are operational details listed in Appendix D.

5 Empirical Methodology

The current evaluation is designed as a transparent empirical record rather than a controlled ablation study. Recent research-agent benchmarks instead hold tasks, evaluators, and time or resource budgets fixed to support direct comparisons (Chan et al., 2024; Wijk et al., 2024; Wang et al., 2026; Lupidi et al., 2026). The present evidence asks whether the deployed runtime has produced competitive artifacts across heterogeneous settings, how broadly it has been used, and how much of the public record is independently reconstructable from committed provenance.

5.1 Research Questions

We organize the evidence around three questions.

RQ1: Cross-domain performance. What task-native results has ARGUS produced across distinct benchmark arenas, and how do those results compare with the human or paper-reported references published for each arena?

RQ2: Research breadth. How many de-duplicated manuscripts and drafts has the research pipeline produced, and how are they distributed across research programs?

RQ3: Evidence maturity. Which public claims are supported only by a timestamped website snapshot, and which additionally include committed artifact identifiers, verifier excerpts, and cryptographic digests?

The questions deliberately avoid causal language. The available records do not compare variants of ARGUS under a shared model, task interface, random seed, and resource budget. Consequently, they can establish operational breadth and task-level outcomes, but not the isolated contribution of role separation, checkpointing, review, or any other individual mechanism.

5.2 Evidence Sources

The benchmark record is a snapshot of the six public result cards on the Argus results page (Argus Team, 2026c). It was retrieved on July 14, 2026 with the final URL, HTTP status, and raw-page SHA-256 stored in `evidence/website_results.json`. Each card preserves the arena, protocol, reported result, comparison text, source URL, and corroboration status.

The research-output record is a de-duplicated inventory of the public Argus paper collection (Argus Team, 2026b), stored in `evidence/paper_inventory.json`. It distinguishes manuscripts from drafts, removes duplicate versions, and groups artifacts into six programs. The inventory is not a publication-acceptance record and is not compared with the paper output of other agents.

The two quantitative figures in Section 6 are generated deterministically from these JSON files. Structural diagrams are explanatory illustrations; they are not empirical plots. Figure provenance is summarized in Appendix C.

5.3 Evidence Tiers

All six benchmark values originate from the public website snapshot. A separate corroboration field records whether an additional artifact-digest record is available. Table 3 defines the distinction used in the results section.

Tier	Available evidence	Permitted interpretation
Artifact digest	Public result card plus logical project, artifact identifier, verifier excerpt, and SHA-256 metadata	A reported Argus result tied to a named external artifact record; external bytes are not included in this repository
Website snapshot	Timestamped public card plus source-page hash	A scoped public claim under the printed protocol, not a reproduction from a named commit
External reference	Human record, paper-reported best, or published system value	A comparison target only; never counted as an Argus result

Table 3. Evidence tiers used for public benchmark claims.

An optional Ed25519 signing path exists for isolated-scorer teammate results, but it is off by default and was not used for the results in this report. The reported evidence chain therefore does not rely on result signing.

5.4 Metrics and Comparison Policy

The six arenas use incompatible metrics: global rank, bits per byte, wall-clock time, solve rate, and a source-reported gap value. We do not normalize these values or average them into a single score. Each result is shown in its native unit with its own protocol

and comparison. Lower is better for bits per byte and wall-clock time. For the Arbor comparison card, the source snapshot does not define the direction of the "gap" metric, so the values are quoted without inferring an ordering beyond the source text.

Human records and paper-reported bests are used when the public card provides them. The SOL-ExecBench row instead uses global rank as its headline and retains the source's system comparison as secondary context. No significance test is introduced where the source record does not provide the underlying repeated measurements.

5.5 Portfolio Counting Policy

The paper inventory counts a research artifact once after duplicate versions are removed. "Manuscript" and "draft" are artifact states, not peer-review outcomes. The headline total therefore answers RQ2—how much research material the pipeline produced across programs—but does not measure novelty, correctness, acceptance, or scientific impact. Those questions require external review beyond the scope of the present evidence bundle.

6 Results

6.1 RQ1: Cross-Domain Performance

Figure 2 and Table 4 report the six public benchmark cards in their native units. The results span kernel optimization, small-language-model training, training-speed optimization, research-assistant tasks, and data synthesis. Because the protocols differ, the figure uses separate panels and the table carries the protocol beside every value.

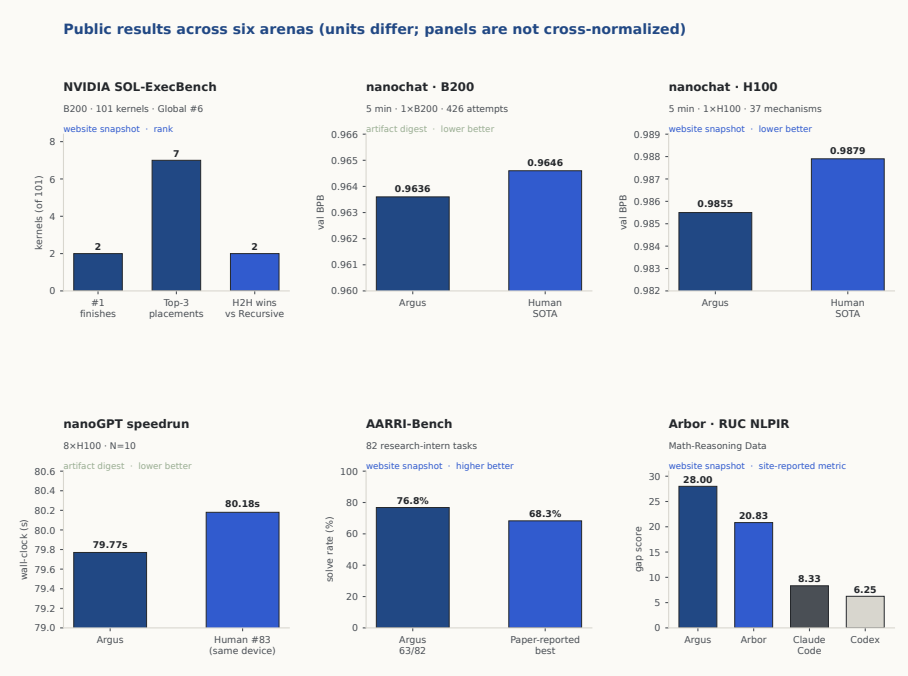

Figure 2. Public results across six arenas. Each panel retains its task-native metric and evidence status; values are not cross-normalized. The figure is generated from `evidence/website_results.json`.

Arena	Protocol	Argus result	Published comparison	Evidence
NVIDIA SOL-ExecBench	B200; 101 kernels	Global #6; 2×#1; 7 top-3	Global rank; two head-to-head wins over Recursive	snapshot
nanochat (B200)	5 min; 1×B200; 426 attempts	0.9636 BPB	Human SOTA 0.9646	**digest**
nanochat (H100)	5 min; 1×H100; 37 mechanisms	0.9855 BPB	Human SOTA 0.9879	snapshot
nanoGPT speedrun	8×H100; N=10	79.77 s	Same-device human #83: 80.18 s	**digest**
AARRI-Bench	82 research-intern tasks	63/82 (76.8%)	Paper-reported best 68.3%	snapshot
Arbor (RUC NLPIR)	Math-reasoning data	28.0 gap	Arbor 20.83; Claude Code 8.33; Codex 6.25	snapshot

Table 4. Six public benchmark results (Argus Team, 2026c). "Digest" means that the website claim additionally has committed artifact-digest metadata; "snapshot" means that the present repository preserves the public card and page hash only.

Several rows meet or exceed the reference printed on the corresponding public card. On nanochat B200, the reported 0.9636

BPB is lower than the listed human SOTA of 0.9646 (Karpathy, 2025). On the nanoGPT speedrun, 79.77 seconds is below the same-device human record of 80.18 seconds (Karpathy, 2022). AARRI-Bench reports 63 of 82 tasks (76.8%), compared with the card's paper-reported best of 68.3% (Wang et al., 2026). The nanochat H100 value of 0.9855 BPB is lower than the listed 0.9879 reference. The SOL-ExecBench headline is Global #6 across 101 kernels, with two first-place head-to-head outcomes and seven top-three outcomes, including the source-reported comparison with Recursive (Recursive, 2026); KernelBench provides the broader public benchmark context for generated GPU kernels (Ouyang et al., 2025). The Arbor row is retained as a source-verbatim system comparison (Jin et al., 2026) because the snapshot does not define the direction of its gap metric.

These outcomes answer RQ1 at the level supported by the public record: ARGUS has produced competitive task-native artifacts across multiple research settings. They do not show that one common model or budget was used across all arenas, and they do not identify which runtime mechanism caused each result.

6.2 RQ2: Research Breadth

The public research inventory contains **41** de-duplicated artifacts: **35 manuscripts** and **6 drafts** across six programs. The distribution is shown in Figure 3 and Table 5. Multimodal and vision-language research is the largest program with 16 artifacts, followed by cognitive-bias studies with 9 and efficiency, compression, and decoding with 7.

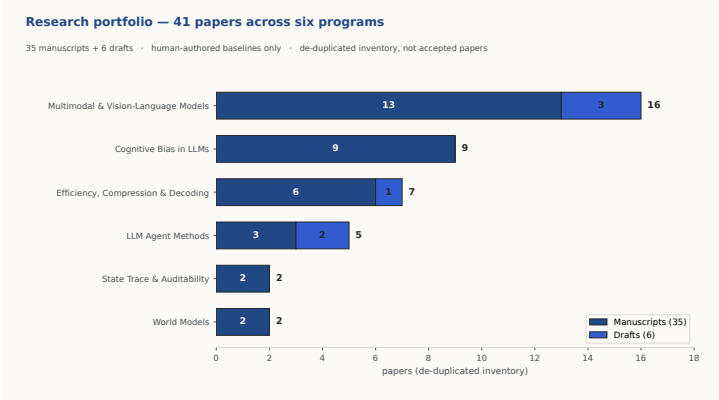

Figure 3. De-duplicated research-artifact inventory by program and status. The chart is generated from `evidence/paper_inventory.json`; manuscript and draft labels do not imply external acceptance.

Program	Manuscripts	Drafts	Total
Multimodal & Vision-Language Models	13	3	16
Cognitive Bias in LLMs	9	0	9
Efficiency, Compression & Decoding	6	1	7
LLM Agent Methods	3	2	5
World Models	2	0	2
State Trace & Auditability	2	0	2
Total	**35**	**6**	**41**

Table 5. Research portfolio by program and status (Argus Team, 2026b).

The inventory provides evidence of sustained output and breadth across task classes. It does not establish that all artifacts are novel, correct, accepted, or of equal quality. The count is therefore reported as a production record rather than a publication metric.

6.3 RQ3: Evidence Maturity

Two of the six benchmark rows have artifact-digest corroboration. The nanoGPT speedrun record identifies an $N=10$ verifier-certified artifact with a mean training time of 79.77 ± 0.06 seconds and a valid seal. The nanochat B200 record identifies a frozen-scorer artifact with `MEAN_VAL_BPB=0.963634`, scorer exit code 0, and a valid SM100 flash-attention implementation. For both rows, the repository stores logical project names, artifact identifiers, verifier excerpts, and SHA-256 values, but not the external artifact bytes.

The other four rows—SOL-ExecBench, nanochat H100, AARRI-Bench, and Arbor—remain website snapshots. Their public values are preserved with a retrieval timestamp and page hash, but they are not currently reproducible from a named commit and artifact bundle. Thus the answer to RQ3 is mixed: the reporting pipeline distinguishes evidence strength correctly, but provenance coverage is incomplete. Completing the artifact chain for all six arenas is the highest-priority evaluation improvement.

7 Analysis and Discussion

7.1 Continuity as a Systems Property

The principal design claim of ARGUS is that long-horizon research should be organized around durable state and bounded work units, not one indefinitely growing conversation. This changes where continuity lives. Recent context remains useful inside a short session, but the campaign objective, current state, artifacts, evidence, and verdicts are external objects with explicit owners. A session can therefore be replaced without erasing the research program.

Role separation addresses a different failure mode. The Engineer is optimized for making progress and may have incentives to frame an outcome favorably. Requiring a Reviewer to issue the completion verdict does not remove model error, but it forces acceptance to occur at a named interface with access to artifacts and execution evidence. This is closer to a reproducible research process than accepting the builder's final summary as the ground truth.

7.2 The Process as a Retained Asset

Equation 8 states that a final artifact contains less information than the run that produced it. The process record includes states, actions, evidence, review feedback, and runtime-state changes, including failed branches that never appear in $y^\star$. This additional information can reduce repeated search, support later audits, and provide training or evaluation material for future systems. ARGUS retains typed, credential-redacted observations and verdicts, not private chain-of-thought transcripts. The distinction matters both for privacy and for scientific interpretation: the durable record documents what was done and shown, not every latent reason the model may have had.

7.3 Evidence-Gated Frontier Expansion

The intended long-term effect of runtime evolution is to enlarge the set of capabilities that can be brought to a new task. Equation 6 expresses both the evidence-admission rule and the desired width/depth behavior. It is not an empirical law. A mission may add no capability, and an admitted skill can later become stale, regress on a harder instance, or be retired. Practical capability is therefore not guaranteed to be monotone. Negative results still have value when they record a counterexample or failed procedure that later missions can avoid, even when $C_{t+1} = C_t$.

This qualified view also clarifies the public evidence. The benchmark rows mainly measure depth on well-defined tasks, while the research inventory measures breadth across programs. The report does not combine them into a universal frontier score. Breadth, depth, provenance quality, and resource efficiency remain separate axes.

7.4 Density Is Not Research Value

Equation 7 prevents a high-throughput system from treating activity as the objective. Dense work contributes little when it produces no new information, fails verification, or cannot be reused. The current evidence bundle measures final task outcomes and provenance coverage, but it does not estimate ΔI, p_{valid}, or η_{reuse} as calibrated time series. A direct empirical test of the website formalization would instrument these terms and compare equal-budget trajectories across systems.

7.5 What the Current Results Support

The results support three modest conclusions. First, the system is operational over heterogeneous tasks rather than demonstrated on a single benchmark family. Second, it can produce artifacts that are competitive with the references printed by the corresponding public records. Third, its evidence pipeline can distinguish stronger artifact-linked claims from weaker public snapshots without silently presenting the latter as reproduced experiments.

The results do not yet support architectural attribution. A stronger study would hold the base model, task interface, budget, evaluator, and randomization fixed while ablating persistent checkpoints, role separation, evidence-log access, skill retention, and planning. Such experiments would test whether the proposed runtime causes higher held-out performance, faster convergence, fewer repeated failures, or better recovery after interruption, following the controlled and trajectory-aware evaluation practices used by AgentBoard, MLE-bench, and RE-Bench (Ma et al., 2024; Chan et al., 2024; Wijk et al., 2024). The current report establishes the system and its public evidence base; controlled causal evaluation remains future work.

8 Limitations and Future Work

No controlled component attribution. The six benchmark records were produced under different task protocols, hardware, and research histories. They show task-level outcomes but do not estimate the causal contribution of the runtime relative to a single-agent baseline. Controlled ablations under common budgets are required before claiming that role separation, bounded memory, or evidence gating improves performance.

Fallible completion authority. The Reviewer is the sole mission-level completion authority, and it is itself a model. It can accept an incomplete result or reject a finished one. Shared artifacts, execution-log access, structured checklists, and fail-closed handling of reviewer backend failures reduce ambiguity, but they do not provide an independent oracle. Future work should

measure Reviewer calibration and study independent evidence checks that strengthen verdict quality without creating conflicting completion authorities.

Incomplete provenance. Only two of the six public benchmark results currently have committed artifact-digest metadata; four are website snapshots. Even the digest-backed rows reference external artifact bytes that are not committed in this repository. Publishing retrievable bundles tied to named commits is necessary for full third-party reproduction.

Benchmark-specific integrity. Randomization, execution-log audits, and artifact inspection can prevent known forms of metric gaming, but every new benchmark introduces a different exploit surface. A leaked evaluation set, an unrandomized input distribution, or an unobserved side channel may defeat existing checks. Measurement integrity must be reassessed per arena.

Model and resource dependence. ARGUS relies on external coding-and-reasoning backends and inherits their capability, latency, availability, and pricing. Reservation-backed budgets bound spend but do not make continuous operation inexpensive. Performance also depends on access to task-specific compute such as GPUs, and the current GPU lease is an explicit operator- or agent-invoked tool rather than an automatic scheduler.

Runtime state can degrade. Persistent memory, skills, verifiers, and routing decisions can be wrong or become stale. Their inspectable and versioned form makes revision possible, but does not guarantee that the effective capability state improves monotonically. Longitudinal evaluation should track regressions, retirements, repeated failures, and the useful lifetime of retained procedures.

Research-output counts are not quality measures. The 41-artifact inventory records production breadth, not peer-review acceptance, novelty, correctness, or impact. External expert review and artifact-level replication are needed to assess scientific quality.

The immediate evaluation priorities are therefore: (1) convert all public results to commit- and artifact-linked bundles; (2) run controlled ablations against common agent baselines; (3) measure interruption recovery, repeated-work reduction, and Reviewer calibration; and (4) report longitudinal capability regressions alongside improvements.

9 Conclusion

We introduced ARGUS, a persistent runtime for long-horizon autonomous research. The system decomposes a campaign into bounded missions, separates planning, execution, and acceptance across explicit roles, records an append-only evidence trace, and retains verified experience as inspectable runtime state around a fixed model. This organization moves continuity from an unbounded model transcript into recoverable project state.

The current public record spans six benchmark arenas and 41 de-duplicated research artifacts across six programs. These results demonstrate operational breadth and competitive task-level outcomes, while the evidence-tier analysis makes clear that only two benchmark rows currently have artifact-digest corroboration. The next step is not a broader headline claim, but stronger causal and reproducibility evidence: common-budget ablations, complete artifact bundles, calibrated review, and longitudinal measurements of what the runtime actually retains.

A Implementation Interfaces and Lifecycle

This appendix records source-level contracts and operational defaults that are useful for reproduction but too detailed for the main argument. The lifecycle is specified directly through role contracts, outcome states, persistence surfaces, and guard conditions; no additional conceptual diagram is required.

A.1 Role Contracts

The Manager front door uses one grounded model decision to emit six structured axes: `CONFIG`, `CONTROL`, `ROUTE`, `LIFETIME`, `FAST_REPLY`, and `NAME`. A `CONTROL: NO_DISPATCH` result is authoritative and fails closed: the bridge may provide a read-only response but may not enqueue work. The Manager is also the sole writer of the current campaign stage.

The Planner produces `TaskSpec` objects with explicit scope, optional DAG dependencies, and impact score. It authors and applies stage `checklist_ops`; the Reviewer can send `checklist_feedback` but does not edit the checklist. The Engineer receives reservation-derived provider fences through `RunnerOptions` and returns a `RunnerResult` with a call identifier, usage-presence flags, cost, and pricing status. The Reviewer returns a `ReviewDecision` whose central fields are listed in Table 6.

Reviewer field	Meaning
`status`	`done`, `continue`, or `blocked`; the mission-level completion verdict
`progress_class`	`decision`, `evidence`, `setup_only`, `artifact_sync_only`, or `none`
`next_action`	Concrete action for the next Engineer round
`operator_question`	At most one human-facing blocking question
`scope, checklist`	Structured final-submission gate, evaluated fail-closed
`skill_ops, wiki_ops`	Reviewer-authored retained-state edits
`backend_unavailable`	Infrastructure failure, never interpreted as a scientific `continue`

Table 6. Abridged Reviewer interface from `core/models.py` and `reviewer/reviewer_schema.json`.

A.2 Outcome Taxonomy

Every mission produces an explicit outcome:

- **Terminal:** `done`, `max_rounds`, `blocked`, `no_progress`, `error`, and `budget_exhausted`.
- **Paused:** `paused_budget`, `paused_provider_cooldown`, `paused_provider_fence`, `infra_blocked`, `research_incomplete`, `paused_no_breakthrough`, and `exhausted_current_methods`.
- **Control transfer:** `replan_requested`; the item returns to pending and passes through the normal Planner gate rather than being counted as completed or failed.

B Persistence and Context Management

The global state root defaults to `~/.argus-skill/` and can be overridden by `ARGUS_SKILL_HOME`. Each project is stored under a fingerprinted subdirectory. Table 7 lists the principal files.

Path	Owner	Content	Mutation
`events.jsonl`	evidence plane	Canonical typed-event timeline	append-only
`backlog.jsonl`	scheduler	Pending, running, completed, and superseded tasks	atomic claim / CAS
`continuous.json`	Manager / scheduler	Standing objective, route, lifetime, and identity	atomic rewrite
`inbox.jsonl`	operator	Operator messages and answers	append + drain
`checkpoint.json`	Reviewer	Hard-capped cross-session working state	atomic rewrite
`daemon.pid`	daemon	Per-project concurrency lock	exclusive
`daemon.status.json`	daemon	Liveness and handoff status	rewrite
`skills/`	Reviewer / operator	Versioned reusable skills	versioned

Table 7. Principal persisted state. The event tape is the canonical timeline; other files are working state or materialized views.

The checkpoint contains the goal, verified work, tried-and-failed approaches, one open blocker, the next step, and bounded environment facts. The Engineer proposes a handoff, and the Reviewer validates and writes the canonical `checkpoint.json`. Python-level item and character caps prevent it from becoming an append-only transcript. A missing or malformed candidate checkpoint preserves the last valid state rather than clearing memory.

By default, each role thread is rolled after three rounds on that thread or when the preceding call reports at least 1.5 million input tokens. The next session is re-seeded from the checkpoint. These limits are configurable and disable at zero. The design keeps provider prefix caching useful over short windows while preventing unbounded compaction loops.

Unattended execution is wrapped by `daemon/life_worker.py` (roughly 1,800 lines). The worker drains on `SIGTERM` or `SIGINT` at a mission boundary. Source upgrades use a blue/green self-handoff with a bounded generation count and persisted identity consisting of the objective hash, vertical, and lineage generation. A resume adopts only a matching identity; otherwise the Manager must perform a fresh division.

C Evidence, Provenance, and Public Records

C.1 Event Catalog and Call Correlation

The canonical catalog contains **112 typed event types** across **11 categories**; **75** event types carry validated payload schemas in `core/event_catalog.py`. Table 8 gives the category counts. The event `research.achievement.certified` is the sole authoritative record of a verified achievement.

Category	Count	Representative content
Lifecycle	40	Loop, round, mission, and budget-reservation events
Skill	20	Match, distillation, use, and outcome events
Wiki	17	Source ingest, page synthesis, promotion, and validation
Planner	9	Planning start, tasks, waiting contracts, and verdicts
Research	7	Achievement certification and research status
Agent I/O	5	Provider-call start, stream, and completion
Daemon	5	Start, stop, handoff, and status
Idea	3	Idea lifecycle
Provider	3	Provider request start, completion, and denial
Usage	2	Presence-aware usage and cost
Operator	1	Operator inbox event

Table 8. Event taxonomy in the current implementation.

Every provider call receives a `call_id` at `core/run_gateway.py`. The identifier is written to the event tape and returned in the runner result, allowing Reviewer prompts and later audits to locate the exact execution record. Usage-presence booleans distinguish a genuine zero from provider-omitted accounting.

C.2 Credential Handling and Optional Signing

`core/secret_guard.py` redacts authorization headers, API-key patterns, provider tokens, and generic key/value secrets before event or artifact copies are persisted for downstream use. A redaction emits `round.secret_redacted`; a scan error or uncovered oversized artifact blocks unconditional certification.

An optional Ed25519 signing path in `team/result_provenance.py` supports isolated-scorer teammate results. It is off by default, is not used for the results reported here, and is not part of their evidence chain.

C.3 Artifact-Digest Records

The nanoGPT speedrun row points to logical project `nanogpt-speedrun-h100` and artifact `candidate_mlp_post_only_hybrid_n10_20260617T152904Z`. Its verifier records `valid=true`, $N{=}10$, validation loss 3.2776, one-sided $p(\text{mean} < 3.28) = 0.004007$, training time 79.77 ± 0.06 seconds, and `seal=ok`. The nanochat B200 row points to logical project `nanochat-mission-b200` and artifact `a374_tailweighted_boundary_mixed_readout_data_bundle`; its frozen scorer exits 0 with `MEAN_VAL_BPB=0.963634` and a valid SM100 flash-attention kernel. The evidence JSON stores source/result hashes and verifier excerpts, but not the external bytes.

Both public pages were retrieved at `2026-07-14T12:29:51Z`. Their complete raw-HTML SHA-256 values are recorded in `evidence/website_results.json` and `evidence/paper_inventory.json` rather than typeset inline.

C.4 Source Map

Subsystem	Primary source path(s)
Runtime composition and roles	`manager/`, `planner/`, `engineer/`, `reviewer/`, `loop.py`
Mission lifecycle and daemon	`life/supervisor/_core.py`, `daemon/life_worker.py`, `daemon/handoff.py`
State and memory	`core/paths.py`, `apps/_runtime.py`, `engineer/checkpoint.py`, `life/memory.py`
Execution and budgets	`core/run_gateway.py`, `core/pricing.py`, `core/usage.py`, `core/provider_quota.py`
Reliability guards	`engineer/runner.py`, `regime_jump/`
Evidence and integrity	`core/event_catalog.py`, `core/secret_guard.py`, `skills/provenance.py`
Cockpit and API	`webapi/server.py`, `frontend/tui/`, `frontend/web/`
Public evidence	`technical_report/evidence/website_results.json`, `paper_inventory.json`

Table 9. Repository map for implementation and evidence claims.

D Reliability, Budgets, and Deployment

The guards in Table 10 define the long-running execution envelope. They are task-agnostic: they constrain activity and resource use, while the Reviewer remains responsible for scientific judgment.

Guard	Trigger and action	Default	Live-call interrupt
Backend failure	Consecutive failures → backoff, then error	2 / 15 s	no
No-progress bail	Consecutive rounds with no Engineer message	2	no
Decision stall	Consecutive nondecision review classes → end	4	no
Decision-progress budget	Time since last decision/evidence → end	1,800 s	no
Soft round escalation	Ask Reviewer to return `blocked`	12	no
Hard round escalation	Force terminal `blocked`	24	no
Effective-progress watchdog	No file change or non-token event	3,600 s	yes
Runner hard-idle	No runner activity	3,600 s	yes
In-round compaction limit	Repeated automatic compactions	3	yes

Table 10. Default reliability guards in `engineer/runner.py`. Thresholds are environment-overridable and disable at zero where applicable.

The canonical `resolve_budget_caps` path in `core/knobs.py` resolves a per-mission preflight cap of $30, a per-project daily cap of $180, a global daily cap of $30, and at most two active daemons across projects. A positive global cap is enforced; setting it to zero disables it. Calls reserve budget before execution and settle or release the reservation afterward. Unpriced calls and provider-fence breaches produce explicit blocked events rather than fabricated costs.

Supervised background jobs maintain their own health cadence and report terminal outcomes through the operator inbox. An Engineer may explicitly emit `WAIT_FOR_SUBAGENT` for a healthy self-watched job; otherwise the request falls back to a normal reviewed round. The GPU lease tool is explicit rather than automatic: `gpu_lease run - <cmd>` releases parked devices, records a lease for the command lifetime, and restores the keep-alive only when no competing lease remains.

The intended deployment is a per-project `systemd -user` service. It runs the daemon in the foreground, uses `SIGTERM`, allows up to 1,800 seconds for a clean mission-boundary drain, and restarts under a bounded service policy. Sandboxed builder roles are confined to the project working directory and may not write the global state root or provider configuration. The terminal and web cockpits read the same event tape; mutating commands and sensitive global views require an authorization token when configured.

E Notation and Figure Provenance

Symbol	Meaning
o, y_0, f, B	Campaign objective, initial artifact, evaluator, and resource budget
τ_t	Mission trajectory $(s_t, a_t, y_t, e_t, r_t)$
$\rho_I(T)$	Dense-intelligence density over wall-clock horizon T
$\dot{N}_{\text{tok}}$	Rate of token-mediated reasoning and action
η_r, η_a, η_v	Reasoning, action, and verification conversion factors
H_t	Runtime state $\{M_t, S_t, A_t, V_t, R_t, Q_t\}$
E_t	Evidence admitted from mission t
θ_t	Underlying model parameters, fixed in this report
C_t	Effective retained capability set
$W_t, D_t(d)$	Capability breadth and depth in domain d
ε	Evidence threshold in the idealized capability-admission rule
$V_R(T)$	Conceptual accumulated research value over horizon T
$\Delta I, p_{\text{valid}}, \eta_{\text{reuse}}$	Information gain, validity probability, and reuse efficiency
D_{process}	Retained states, actions, evidence, feedback, and runtime-state deltas

Table 11. Notation used in the main text.

The paper uses three figures, each with a distinct evidentiary purpose. The first-page teaser, `argus_teaser`, combines the exact objective–runtime–evidence–state flow with protocol-scoped public result cards. It is an HTML/CSS vector composition generated from the committed evidence JSON by `figures/build_argus_teaser.py`; its figure brief, source hashes, editable HTML, and renderer are recorded in `figures/argus_teaser.json`. The two empirical figures, `public_results` and `paper_portfolio`, are generated by `figures/build_report_figures.py` and digested in `figures/REPORT_FIGURES.json`. The remaining structural assets in the repository are intentionally not included: their content is represented more precisely by equations, tables, and interface definitions in this version.

References

Argus Team. argus-skill: A 7×24 system for long-horizon autonomous research. GitHub repository, 2026a. URL https://github.com/lbx154/argus-skill.

Argus Team. Argus research-paper collection. Website snapshot, https://argusbot.cn/research.html, 2026b. URL https://argusbot.cn/research.html.

Argus Team. Argus public results. Website snapshot, https://argusbot.cn/results.html, 2026c. URL https://argusbot.cn/results.html.

Argus Team. Argus: A self-evolving research agent. Project website, 2026d. URL https://argusbot.cn/.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, et al. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024. URL https://arxiv.org/abs/2410.07095.

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, et al. AIDE: AI-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025. URL https://arxiv.org/abs/2502.13138.

Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2024. URL https://arxiv.org/abs/2310.06770.

Jiajie Jin, Yuyang Hu, Kai Qiu, Qi Dai, Chong Luo, et al. Toward generalist autonomous research via hypothesis-tree refinement. *arXiv preprint arXiv:2606.11926*, 2026. URL https://arxiv.org/abs/2606.11926.

Andrej Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. GitHub repository, 2022. URL https://github.com/karpathy/nanoGPT.

Andrej Karpathy. nanochat: The best ChatGPT that $100 can buy. GitHub repository, 2025. URL https://github.com/karpathy/nanochat.

Thomas Kwa, Ben West, Joel Becker, et al. Measuring AI ability to complete long software tasks. *arXiv preprint arXiv:2503.14499*, 2025. URL https://arxiv.org/abs/2503.14499.

Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023. URL https://arxiv.org/abs/2308.03688.

Chris Lu et al. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024. URL https://arxiv.org/abs/2408.06292.

Alisia Lupidi, Bhavul Gauri, Thomas Simon Foster, et al. AIRS-Bench: A suite of tasks for frontier AI research science agents. *arXiv preprint arXiv:2602.06855*, 2026. URL https://arxiv.org/abs/2602.06855.

Chang Ma, Junlei Zhang, Zhihao Zhu, et al. AgentBoard: An analytical evaluation board of multi-turn LLM agents. *arXiv preprint arXiv:2401.13178*, 2024. URL https://arxiv.org/abs/2401.13178.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. URL https://arxiv.org/abs/2506.13131.

Anne Ouyang et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025. URL https://arxiv.org/abs/2502.10517.

Charles Packer, Sarah Wooders, Kevin Lin, et al. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, et al. Generative agents: Interactive simulacra of human behavior. *arXiv preprint arXiv:2304.03442*, 2023. URL https://arxiv.org/abs/2304.03442.

Recursive. First steps toward automated AI research. GitHub repository, 2026. URL https://github.com/recursive-org/first-steps-toward-automated-ai-research.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. URL https://arxiv.org/abs/2302.04761.

Samuel Schmidgall, Yusheng Su, Ze Wang, et al. Agent laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025. URL https://arxiv.org/abs/2501.04227.

Noah Shinn et al. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023. URL https://arxiv.org/abs/2303.11366.

Guanzhi Wang et al. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023. URL https://arxiv.org/abs/2305.16291.

Jiayu Wang, Weijiang Lv, Bowen Fu, et al. Act as a real researcher: A suite of benchmarks evaluating frontier LLMs and agentic harnesses in research lifecycle. *arXiv preprint arXiv:2606.07462*, 2026. URL https://arxiv.org/abs/2606.07462.

Xingyao Wang, Boxuan Li, Yufan Song, et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024a. URL https://arxiv.org/abs/2407.16741.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024b. URL https://arxiv.org/abs/2409.07429.

Hjalmar Wijk, Tao Lin, Joel Becker, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024. URL https://arxiv.org/abs/2411.15114.

Wujiang Xu, Zujie Liang, Kai Mei, et al. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025. URL https://arxiv.org/abs/2502.12110.

Yutaro Yamada, Robert Tjarko Lange, Cong Lu, et al. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025. URL https://arxiv.org/abs/2504.08066.

John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. URL https://arxiv.org/abs/2405.15793.

Shunyu Yao et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023. URL https://arxiv.org/abs/2210.03629.